Chapter 7. Developing Robust Programs

This chapter deals with developing programs that are *robust*. Robust programs are able to handle error conditions gracefully. They are programs that do not crash no matter what the user does. Building robust programs is essential to the practice of programming. Writing robust programs takes discipline and work - it usually entails finding every possible problem that can occur, and coming up with an action plan for your program to take.

Where Does the Time Go?

Programmers schedule poorly. In almost every programming project, programmers will take two, four, or even eight times as long to develop a program or function than they originally estimated. There are many reasons for this problem, including:

- Programmers don’t always schedule time for meetings or other non-coding activities that make up every day.
- Programmers often underestimate feedback times (how long it takes to pass change requests and approvals back and forth) for projects.
- Programmers don’t always understand the full scope of what they are producing.
- Programmers often have to estimate a schedule on a totally different kind of project than they are used to, and thus are unable to schedule accurately.
- Programmers often underestimate the amount of time it takes to get a program fully robust.

The last item is the one we are interested in here. *It takes a lot of time and effort to develop robust programs.* More so than people usually guess, including

experienced programmers. Programmers get so focused on simply solving the problem at hand that they fail to look at the possible side issues.

In the `toupper` program, we do not have any course of action if the file the user selects does not exist. The program will go ahead and try to work anyway. It doesn't report any error message so the user won't even know that they typed in the name wrong. Let's say that the destination file is on a network drive, and the network temporarily fails. The operating system is returning a status code to us in `%eax`, but we aren't checking it. Therefore, if a failure occurs, the user is totally unaware. This program is definitely not robust. As you can see, even in a simple program there are a lot of things that can go wrong that a programmer must contend with.

In a large program, it gets much more problematic. There are usually many more possible error conditions than possible successful conditions. Therefore, you should always expect to spend the majority of your time checking status codes, writing error handlers, and performing similar tasks to make your program robust. If it takes two weeks to develop a program, it will likely take at least two more to make it robust. Remember that every error message that pops up on your screen had to be programmed in by someone.

Some Tips for Developing Robust Programs

User Testing

Testing is one of the most essential things a programmer does. If you haven't tested something, you should assume it doesn't work. However, testing isn't just about making sure your program works, it's about making sure your program doesn't break. For example, if I have a program that is only supposed to deal with positive numbers, you need to test what happens if the user enters a negative number. Or a letter. Or the number zero. You must test what happens if they put spaces before their numbers, spaces after their numbers, and other little

possibilities. You need to make sure that you handle the user's data in a way that makes sense to the user, and that you pass on that data in a way that makes sense to the rest of your program. When your program finds input that doesn't make sense, it needs to perform appropriate actions. Depending on your program, this may include ending the program, prompting the user to re-enter values, notifying a central error log, rolling back an operation, or ignoring it and continuing.

Not only should you test your programs, you need to have others test it as well. You should enlist other programmers and users of your program to help you test your program. If something is a problem for your users, even if it seems okay to you, it needs to be fixed. If the user doesn't know how to use your program correctly, that should be treated as a bug that needs to be fixed.

You will find that users find a lot more bugs in your program than you ever could. The reason is that users don't know what the computer expects. You know what kinds of data the computer expects, and therefore are much more likely to enter data that makes sense to the computer. Users enter data that makes sense to them. Allowing non-programmers to use your program for testing purposes usually gives you much more accurate results as to how robust your program truly is.

Data Testing

When designing programs, each of your functions needs to be very specific about the type and range of data that it will or won't accept. You then need to test these functions to make sure that they perform to specification when handed the appropriate data. Most important is testing *corner cases* or *edge cases*. Corner cases are the inputs that are most likely to cause problems or behave unexpectedly.

When testing numeric data, there are several corner cases you always need to test:

- The number 0
- The number 1

- A number within the expected range
- A number outside the expected range
- The first number in the expected range
- The last number in the expected range
- The first number below the expected range
- The first number above the expected range

For example, if I have a program that is supposed to accept values between 5 and 200, I should test 0, 1, 4, 5, 153, 200, 201, and 255 at a minimum (153 and 255 were randomly chosen inside and outside the range, respectively). The same goes for any lists of data you have. You need to test that your program behaves as expected for lists of 0 items, 1 item, massive numbers of items, and so on. In addition, you should also test any turning points you have. For example, if you have different code to handle people under and over age 30, for example, you would need to test it on people of ages 29, 30, and 31 at least.

There will be some internal functions that you assume get good data because you have checked for errors before this point. However, while in development you often need to check for errors anyway, as your other code may have errors in it. To verify the consistency and validity of data during development, most languages have a facility to easily check assumptions about data correctness. In the C language there is the `assert` macro. You can simply put in your code `assert(a > b);`, and it will give an error if it reaches that code when the condition is not true. In addition, since such a check is a waste of time after your code is stable, the `assert` macro allows you to turn off asserts at compile-time. This makes sure that your functions are receiving good data without causing unnecessary slowdowns for code released to the public.

Module Testing

Not only should you test your program as a whole, you need to test the individual

pieces of your program. As you develop your program, you should test individual functions by providing it with data you create to make sure it responds appropriately.

In order to do this effectively, you have to develop functions whose sole purpose is to call functions for testing. These are called *drivers* (not to be confused with hardware drivers) . They simply loads your function, supply it with data, and check the results. This is especially useful if you are working on pieces of an unfinished program. Since you can't test all of the pieces together, you can create a driver program that will test each function individually.

Also, the code you are testing may make calls to functions not developed yet. In order to overcome this problem, you can write a small function called a *stub* which simply returns the values that function needs to proceed. For example, in an e-commerce application, I had a function called `is_ready_to_checkout`. Before I had time to actually write the function I just set it to return true on every call so that the functions which relied on it would have an answer. This allowed me to test functions which relied on `is_ready_to_checkout` without the function being fully implemented.

Handling Errors Effectively

Not only is it important to know how to test, but it is also important to know what to do when an error is detected.

Have an Error Code for Everything

Truly robust software has a unique error code for every possible contingency. By simply knowing the error code, you should be able to find the location in your code where that error was signalled.

This is important because the error code is usually all the user has to go on when reporting errors. Therefore, it needs to be as useful as possible.

Error codes should also be accompanied by descriptive error messages. However, only in rare circumstances should the error message try to predict *why* the error occurred. It should simply relate what happened. Back in 1995 I worked for an Internet Service Provider. One of the web browsers we supported tried to guess the cause for every network error, rather than just reporting the error. If the computer wasn't connected to the Internet and the user tried to connect to a website, it would say that there was a problem with the Internet Service Provider, that the server was down, and that the user should contact their Internet Service Provider to correct the problem. Nearly a quarter of our calls were from people who had received this message, but merely needed to connect to the Internet before trying to use their browser. As you can see, trying to diagnose what the problem is can lead to a lot more problems than it fixes. It is better to just report error codes and messages, and have separate resources for the user to troubleshooting the application. A troubleshooting guide, not the program itself, is an appropriate place to list possible reasons and courses for action for each error message.

Recovery Points

In order to simplify error handling, it is often useful to break your program apart into distinct units, where each unit fails and is recovered as a whole. For example, you could break your program up so that reading the configuration file was a unit. If reading the configuration file failed at any point (opening the file, reading the file, trying to decode the file, etc.) then the program would simply treat it as a configuration file problem and skip to the *recovery point* for that problem. This way you greatly reduce the number of error-handling mechanism you need for your program, because error recovery is done on a much more general level.

Note that even with recovery points, your error messages need to be specific as to what the problem was. Recovery points are basic units for error recovery, not for error detection. Error detection still needs to be extremely exact, and the error

reports need exact error codes and messages.

When using recovery points, you often need to include cleanup code to handle different contingencies. For example, in our configuration file example, the recovery function would need to include code to check and see if the configuration file was still open. Depending on where the error occurred, the file may have been left open. The recovery function needs to check for this condition, and any other condition that might lead to system instability, and return the program to a consistent state.

The simplest way to handle recovery points is to wrap the whole program into a single recovery point. You would just have a simple error-reporting function that you can call with an error code and a message. The function would print them and and simply exit the program. This is not usually the best solution for real-world situations, but it is a good fall-back, last resort mechanism.

Making Our Program More Robust

This section will go through making the `add-year.s` program from Chapter 6 a little more robust.

Since this is a pretty simple program, we will limit ourselves to a single recovery point that covers the whole program. The only thing we will do to recover is to print the error and exit. The code to do that is pretty simple:

```
 .include "linux.s"
 .equ ST_ERROR_CODE, 8
 .equ ST_ERROR_MSG, 12
 .globl error_exit
 .type error_exit, @function
error_exit:
 pushl %ebp
 movl  %esp, %ebp
```

```
#Write out error code
movl  ST_ERROR_CODE(%ebp), %ecx
pushl %ecx
call  count_chars
popl  %ecx
movl  %eax, %edx
movl  $STDERR, %ebx
movl  $SYS_WRITE, %eax
int   $LINUX_SYSCALL

#Write out error message
movl  ST_ERROR_MSG(%ebp), %ecx
pushl %ecx
call  count_chars
popl  %ecx
movl  %eax, %edx
movl  $STDERR, %ebx
movl  $SYS_WRITE, %eax
int   $LINUX_SYSCALL

pushl $STDERR
call  write_newline

#Exit with status 1
movl  $SYS_EXIT, %eax
movl  $1, %ebx
int   $LINUX_SYSCALL
```

Enter it in a file called `error-exit.s`. To call it, you just need to push the address of an error message, and then an error code onto the stack, and call the function.

Now let's look for potential error spots in our `add-year` program. First of all, we don't check to see if either of our `open` system calls actually complete properly.

Linux returns its status code in `%eax`, so we need to check and see if there is an error.

```
 #Open file for reading
 movl  $SYS_OPEN, %eax
 movl  $input_file_name, %ebx
 movl  $0, %ecx
 movl  $0666, %edx
 int   $LINUX_SYSCALL

 movl  %eax, INPUT_DESCRIPTOR(%ebp)

 #This will test and see if %eax is
 #negative.  If it is not negative, it
 #will jump to continue_processing.
 #Otherwise it will handle the error
 #condition that the negative number
 #represents.
 cmpl  $0, %eax
 jl    continue_processing


 #Send the error
 .section .data
no_open_file_code:
 .ascii "0001: \0"
no_open_file_msg:
 .ascii "Can't Open Input File\0"

 .section .text
 pushl $no_open_file_msg
 pushl $no_open_file_code
 call  error_exit

continue_processing:
```

```
#Rest of program
```

So, after calling the system call, we check and see if we have an error by checking to see if the result of the system call is less than zero. If so, we call our error reporting and exit routine.

After every system call, function call, or instruction which can have erroneous results you should add error checking and handling code.

To assemble and link the files, do:

```
as add-year.s -o add-year.o
as error-exit.s -o error-exit.o
ld add-year.o write-newline.o error-exit.o read-record.o write-
record.o count-chars.o -o add-year
```

Now try to run it without the necessary files. It now exits cleanly and gracefully!

Review

Know the Concepts

- What are the reasons programmer's have trouble with scheduling?
- Find your favorite program, and try to use it in a completely wrong manner. Open up files of the wrong type, choose invalid options, close windows that are supposed to be open, etc. Count how many different error scenarios they had to account for.
- What are corner cases? Can you list examples of numeric corner cases?
- Why is user testing so important?
- What are stubs and drivers used for? What's the difference between the two?

- What are recovery points used for?
- How many different error codes should a program have?

Use the Concepts

- Go through the `add-year.s` program and add error-checking code after every system call.
- Find one other program we have done so far, and add error-checking to that program.
- Add a recovery mechanism for `add-year.s` that allows it to read from STDIN if it cannot open the standard file.

Going Further

- What, if anything, should you do if your error-reporting function fails? Why?
- Try to find bugs in at least one open-source program. File a bug report for it.
- Try to fix the bug you found in the previous exercise.